CS228: Introduction to Deep Learning

Lecture 6: gradient descent

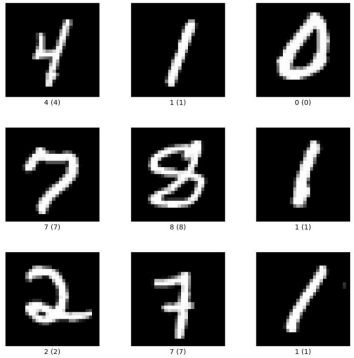
Instructor: Zhouxing Shi

Recap: Softmax

A special activation that is applied to multiple neurons

$$y_i = \mathrm{Softmax}(z) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \qquad \sum_i y_i = 1$$

Model probability P(y=i | x) in multi-class classification settings



Equivalent to sigmoid in binary classification

Recap: KL v.s. cross-entropy

KL divergence: $$\mathrm{KL}(y, g) = \sum_j g_j \log \frac{g_j}{y_j} = \sum_j g_j \log g_j - \sum_j g_j \log y_j$$

Difference when we use y (prediction) to approximate g (ground-truth)

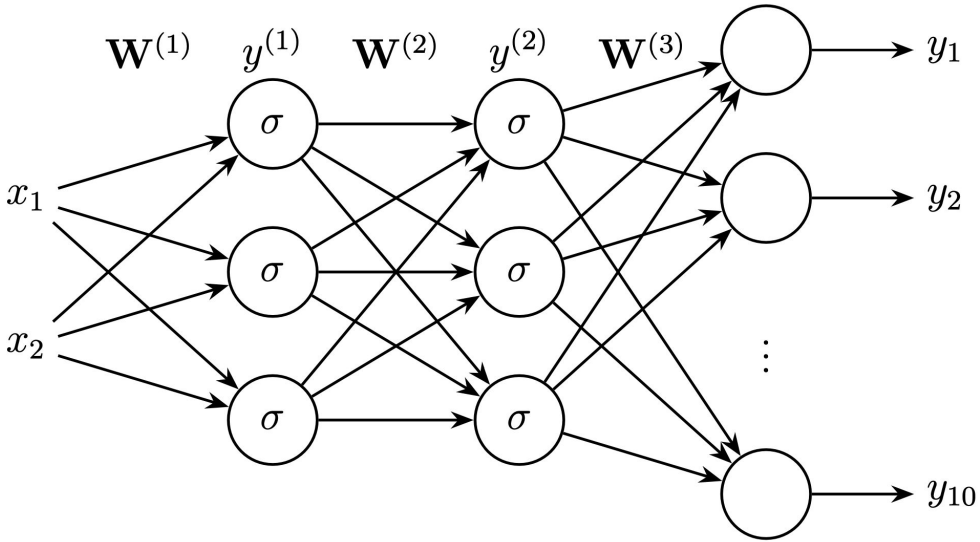
Cross entropy: $$H(y, g) = -\sum_j g_j \log y_j \qquad H(y, g) = \mathrm{KL}(y, g) + H(g, g)$$

Total number of bits in expectation (add entropy of g)

KL and cross entropy are equivalent when g is one-hot

Usually a cross-entropy loss is used for classification problems

“CrossEntropyLoss”



$$L = \text{Cross-entropy}(\text{Softmax}(y), g) = -\sum_j g_j \log \frac{\exp(y_j)}{\sum_k \exp(y_k)} = \underbrace{-\log \frac{\exp(y_c)}{\sum_k \exp(y_k)}}_{\text{one-hot } g \text{ with label } c}$$

Gradient

Partial derivatives:

$$df(X) = \nabla_X f(X) dX$$

$$\nabla_X f(X) = \begin{bmatrix} \frac{\partial f(X)}{\partial x_1} & \frac{\partial f(X)}{\partial x_2} & \cdots & \frac{\partial f(X)}{\partial x_d} \end{bmatrix}$$

Gradient:

$$\nabla_X f(X)^\top$$

Change in f(X) for any tiny variation in each x_i

f(x) changes the fastest when dX aligns with the gradient

Gradient descent

- Repeated update weights using **gradients w.r.t. weights**:

$$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

- α is the step size (learning rate)
  - Too large: diverge
  - Too small: slow convergence

Whiteboard: demo

Gradient descent

- Form an approximation function at each iteration

$$f(\mathbf{W}^{(t)} + \mathbf{d}) \approx h(\mathbf{d}) := f(\mathbf{W}^{(t)}) + \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})^\top d + \frac{1}{2\alpha}\|\mathbf{d}\|^2$$

(regularizes step size)

- Update rule: $\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} + \mathbf{d}^*$ $\quad \mathbf{d}^* = \arg\min_{\mathbf{d}} h(\mathbf{d})$

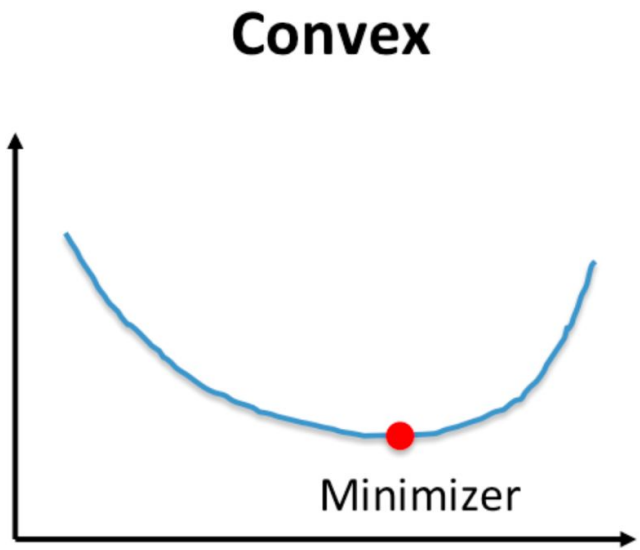
$$\nabla_{\mathbf{d}} g(\mathbf{d}^*) = 0 \Rightarrow \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)}) + \frac{1}{\alpha}\mathbf{d}^* = 0 \qquad \Rightarrow \mathbf{d}^* = -\alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

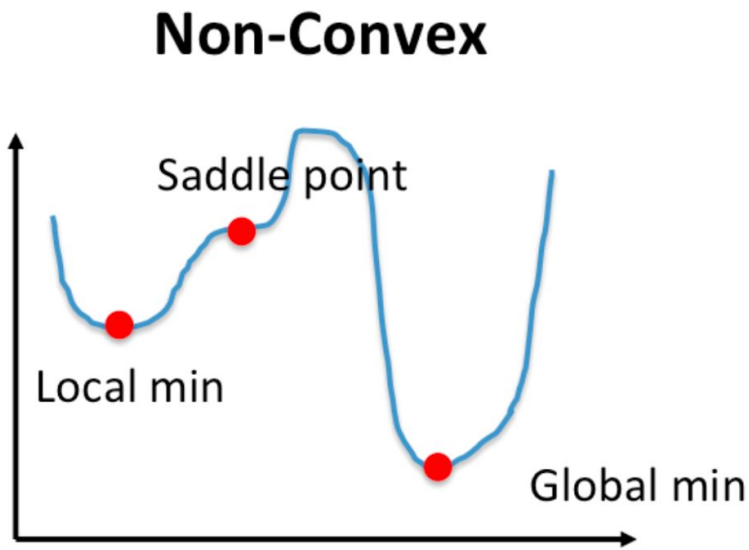
(with sufficiently small α
and h is an upper bound of f)

Whiteboard: illustration (update rule with quadratic approximation)

Gradient descent

For non-convex functions (e.g., neural networks), optimization typically only converges to stationary points (not global minimum)





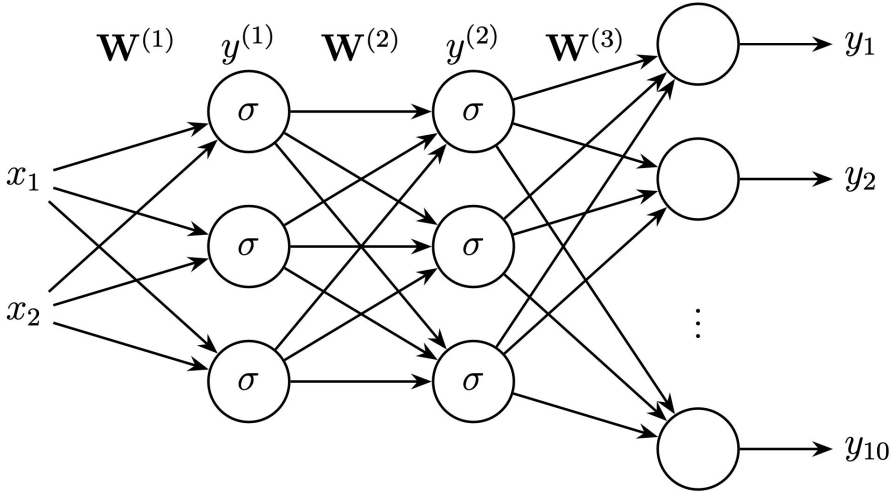
Gradients for the loss function

$$L = \text{Cross-entropy}(\text{Softmax}(y), g) = -\sum_j g_j \log \frac{\exp(y_j)}{\sum_k \exp(y_k)} = \underbrace{-\log \frac{\exp(y_c)}{\sum_k \exp(y_k)}}_{\text{one-hot } g \text{ with label } c}$$

$$\text{Let } p = \text{Softmax}(y),\ p_j = \frac{\exp(y_j)}{\sum_k \exp(y_k)}, \quad L = -\log p_c$$

$$\frac{\partial L}{\partial y_i} = p_i - \mathbf{1}[i = c] \qquad \nabla_y L = p - g$$

Gradients for layers



We have: $\dfrac{\partial L}{\partial y_i}$

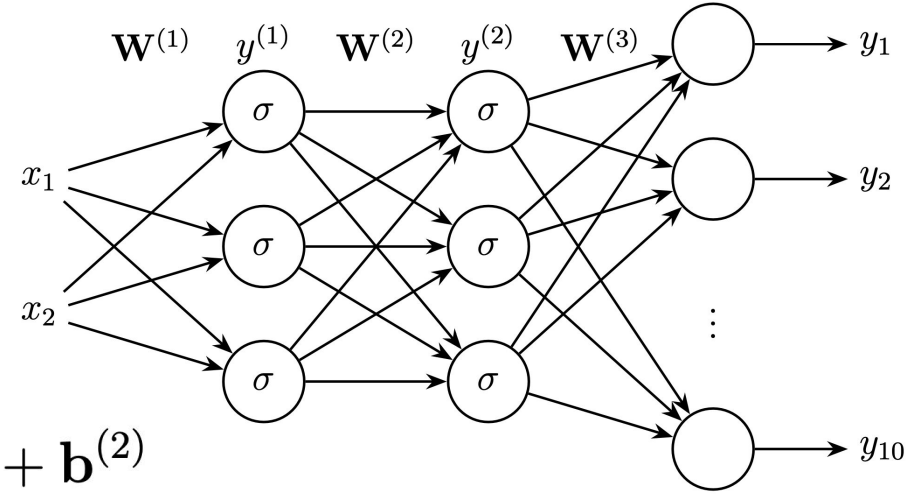
Let $y = \mathbf{W}^{(3)} y^{(2)} + \mathbf{b}^{(3)}$

$$\frac{\partial L}{\partial b_i^{(3)}} = \frac{\partial L}{\partial y_i} = p_i - g_i \qquad \nabla_{\mathbf{b}^{(3)}} L = p - g$$

$$\frac{\partial L}{\partial W_{ij}^{(3)}} = \frac{\partial L}{\partial y_i} \cdot \frac{\partial y_i}{\partial W_{ij}^{(3)}} = (p_i - g_i)\, y_j^{(2)} \qquad \nabla_{\mathbf{W}^{(3)}} L = (p - g)(y^{(2)})^\top$$

Next: $$\frac{\partial L}{\partial y_j^{(2)}} = \sum_i \frac{\partial L}{\partial y_i} \cdot W_{ij}^{(3)} \qquad \nabla_{y^{(2)}} L = (\mathbf{W}^{(3)})^\top (p - g)$$

Gradients for layers



We have: $\dfrac{\partial L}{\partial y_j^{(2)}}$

$$\text{Let } y^{(2)} = \sigma(z^{(2)}),\ z^{(2)} = \mathbf{W}^{(2)} y^{(1)} + \mathbf{b}^{(2)}$$

$$\frac{\partial L}{\partial z_j^{(2)}} = \frac{\partial L}{\partial y_j^{(2)}} \cdot \sigma'(z_j^{(2)}) \qquad \nabla_{z^{(2)}} L = \nabla_{y^{(2)}} L \odot \sigma'(z^{(2)})$$

$$\nabla_{\mathbf{b}^{(2)}} L = \nabla_{z^{(2)}} L \qquad \nabla_{\mathbf{W}^{(2)}} L = \nabla_{z^{(2)}} L \cdot (y^{(1)})^\top$$

Next: $\dfrac{\partial L}{\partial y_j^{(1)}} = \displaystyle\sum_i \frac{\partial L}{\partial z_i^{(2)}} \cdot W_{ij}^{(2)} \qquad \nabla_{y^{(1)}} L = (\mathbf{W}^{(2)})^\top \nabla_{z^{(2)}} L$

Gradients for layers

Given $\nabla_{y^{(l)}} L,\ y^{(l)} = \sigma(z^{(l)}),\ z^{(l)} = \mathbf{W}^{(l)} y^{(l-1)} + \mathbf{b}^{(l)}$

$$\nabla_{z^{(l)}} L = \nabla_{y^{(l)}} L \odot \sigma'(z^{(l)})$$

$$\nabla_{\mathbf{b}^{(l)}} L = \nabla_{z^{(l)}} L$$

$$\nabla_{\mathbf{W}^{(l)}} L = \nabla_{z^{(l)}} L \cdot (y^{(l-1)})^\top$$

$$\nabla_{y^{(l-1)}} L = (\mathbf{W}^{(l)})^\top \nabla_{z^{(l)}} L$$

Gradients get propagated from layer *l* to layer *l-1* in a **backward** manner

Forward and backward passes

Forward $(l = 1, 2, \ldots, n-1)$

$$y^{(n)} = \mathbf{W}^{(n)} y^{(n-1)} + \mathbf{b}^{(n)}$$

$$z^{(l)} = \mathbf{W}^{(l)} y^{(l-1)} + \mathbf{b}^{(l)}$$

$$y^{(l)} = \sigma(z^{(l)})$$

Backward $(l = n-1, n-2, \ldots, 1)$

$$\nabla_{y^{(n)}} L = p - g$$

$$\nabla_{z^{(l)}} L = \nabla_{y^{(l)}} L \odot \sigma'(z^{(l)})$$

$$\nabla_{\mathbf{b}^{(l)}} L = \nabla_{z^{(l)}} L$$

$$\nabla_{\mathbf{W}^{(l)}} L = \nabla_{z^{(l)}} L \cdot (y^{(l-1)})^\top$$

$$\nabla_{y^{(l-1)}} L = (\mathbf{W}^{(l)})^\top \nabla_{z^{(l)}} L$$

Training

For every iteration with training examples:

- Forward pass to get outputs
- Calculate the loss function on the outputs
- Backward pass to get gradients
    - Automatically done by *automatic differentiation (autodiff)*
- Gradient descent to update parameters

Whiteboard: autodiff

Assignment 1

- Assignment 1 to be released next Monday
- Topic: getting started with deep learning coding
- Lecture on Monday will introduce Pytorch coding